

75.03 & 95.57 Organización del Computador

Compiladores y
Ensambladores



Operación

Lenguaje alto nivel

Assembler

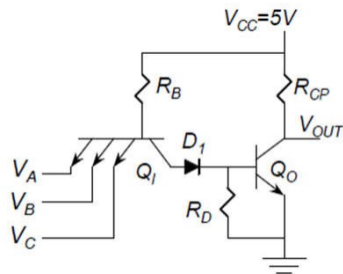
Código de máquina

Procesador

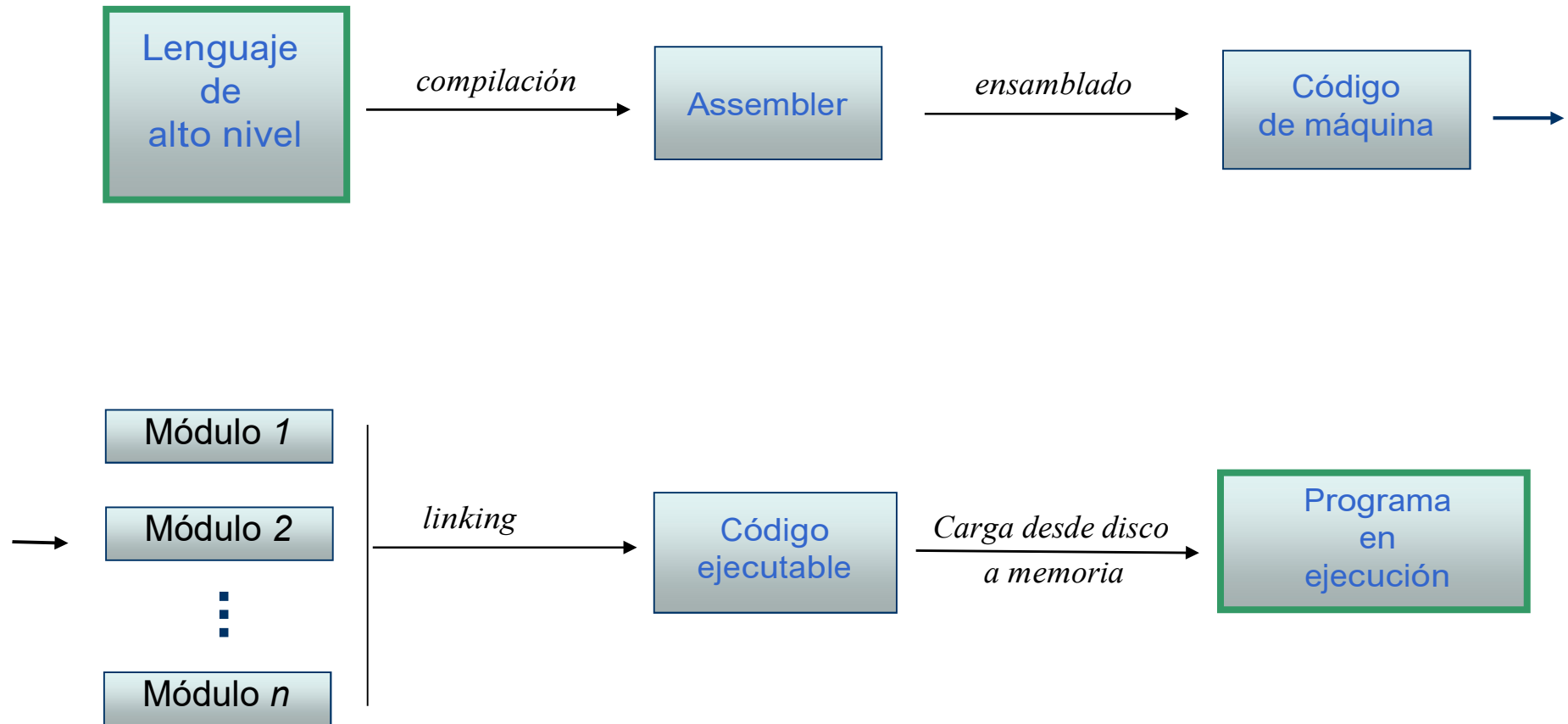
Compuertas, ALU

Transistores
diodos, etc

Niveles de abstracción
en una computadora



Desde el diseño a la ejecución



Compiladores

- Compilador de **una sola pasada o de pasadas múltiples**
Completa el proceso en un solo recorrido del programa fuente o en varios

- **Compilador incremental**

Genera código instrucción por instrucción (no para todo el programa) cuando usuario pulsa una tecla. Entorno de depuración. Intérpretes

- **Cross-compiler**

Genera código en lenguaje assembler para una máquina diferente de la que se está utilizando para compilar.

El proceso de compilación

Datos de
entrada

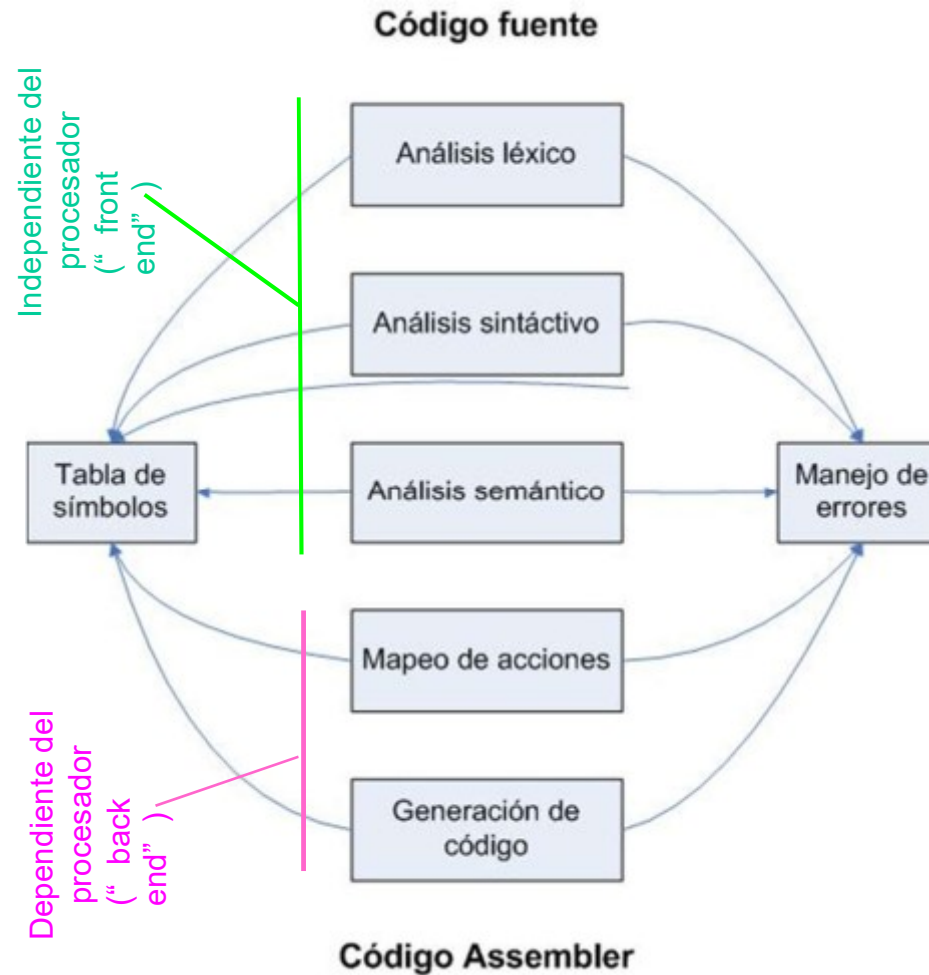
Código fuente



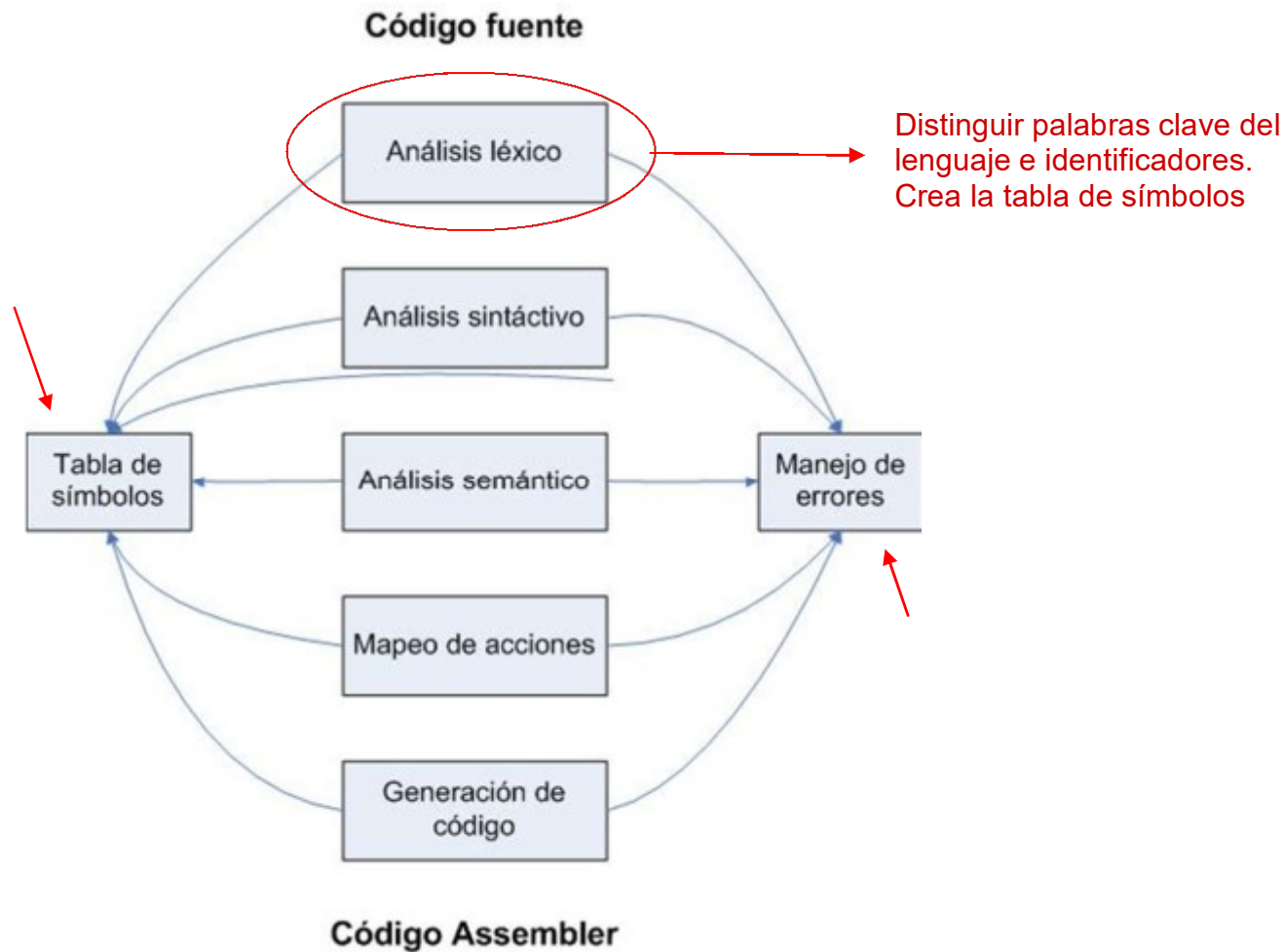
Salida del
proceso

Código Assembler

El proceso de compilación



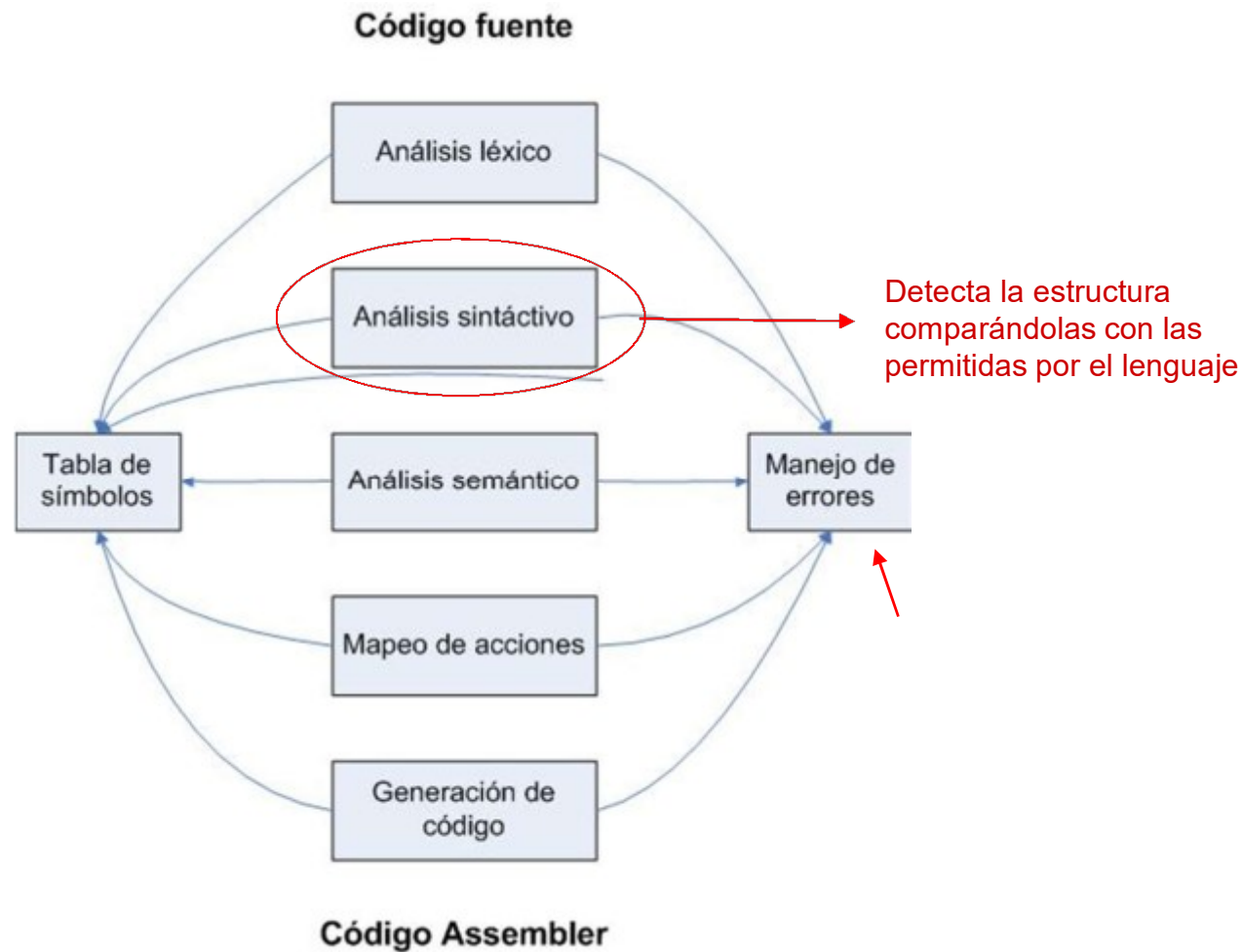
El proceso de compilación



Total = Precio * 5

id1 = id2 * Cte

El proceso de compilación

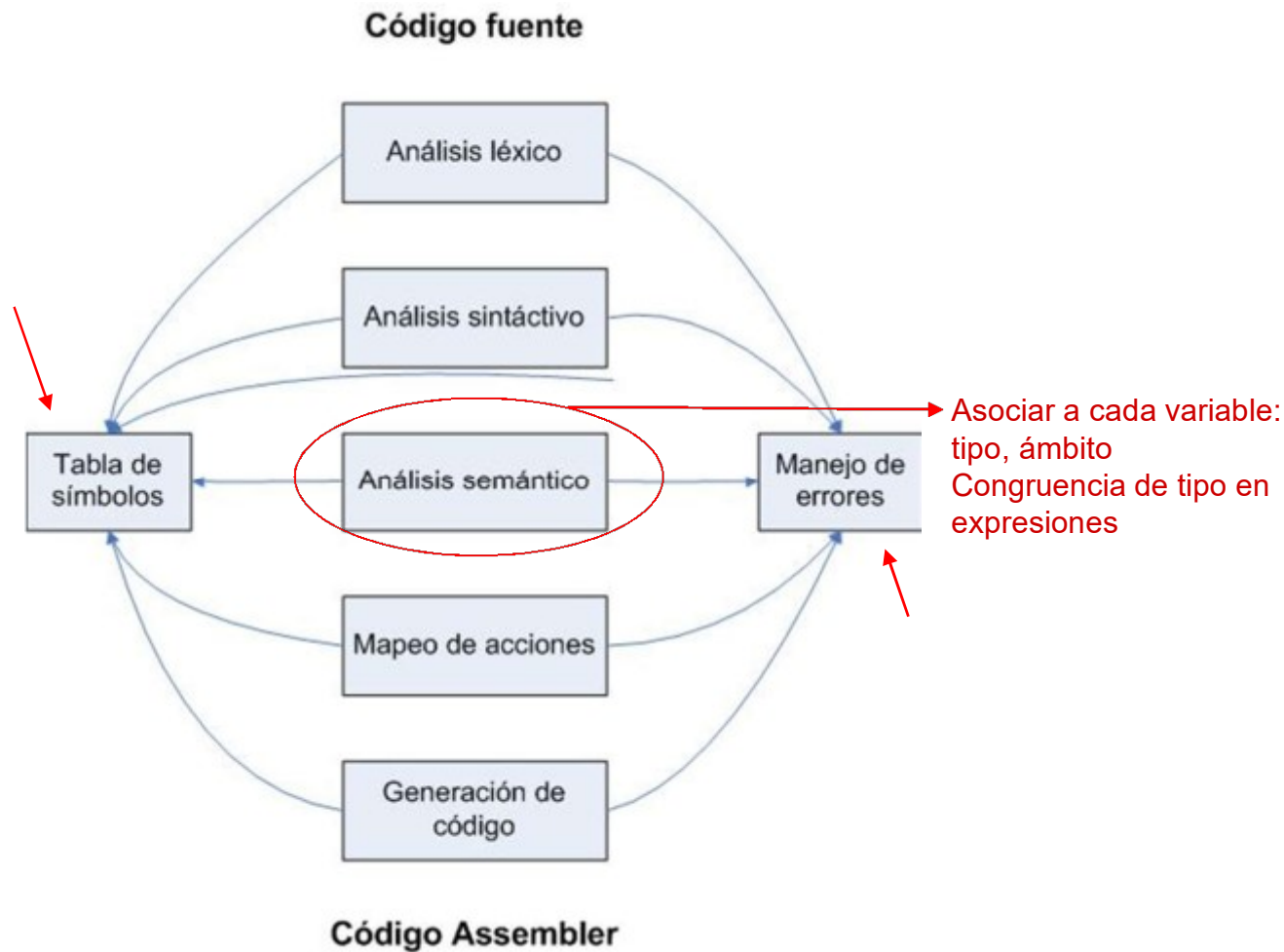


Total = Precio * 5

id1 = id2 * Cte

asignación: id1 "=" expresión
Producto:: "expresión" = id2*Cte

El proceso de compilación



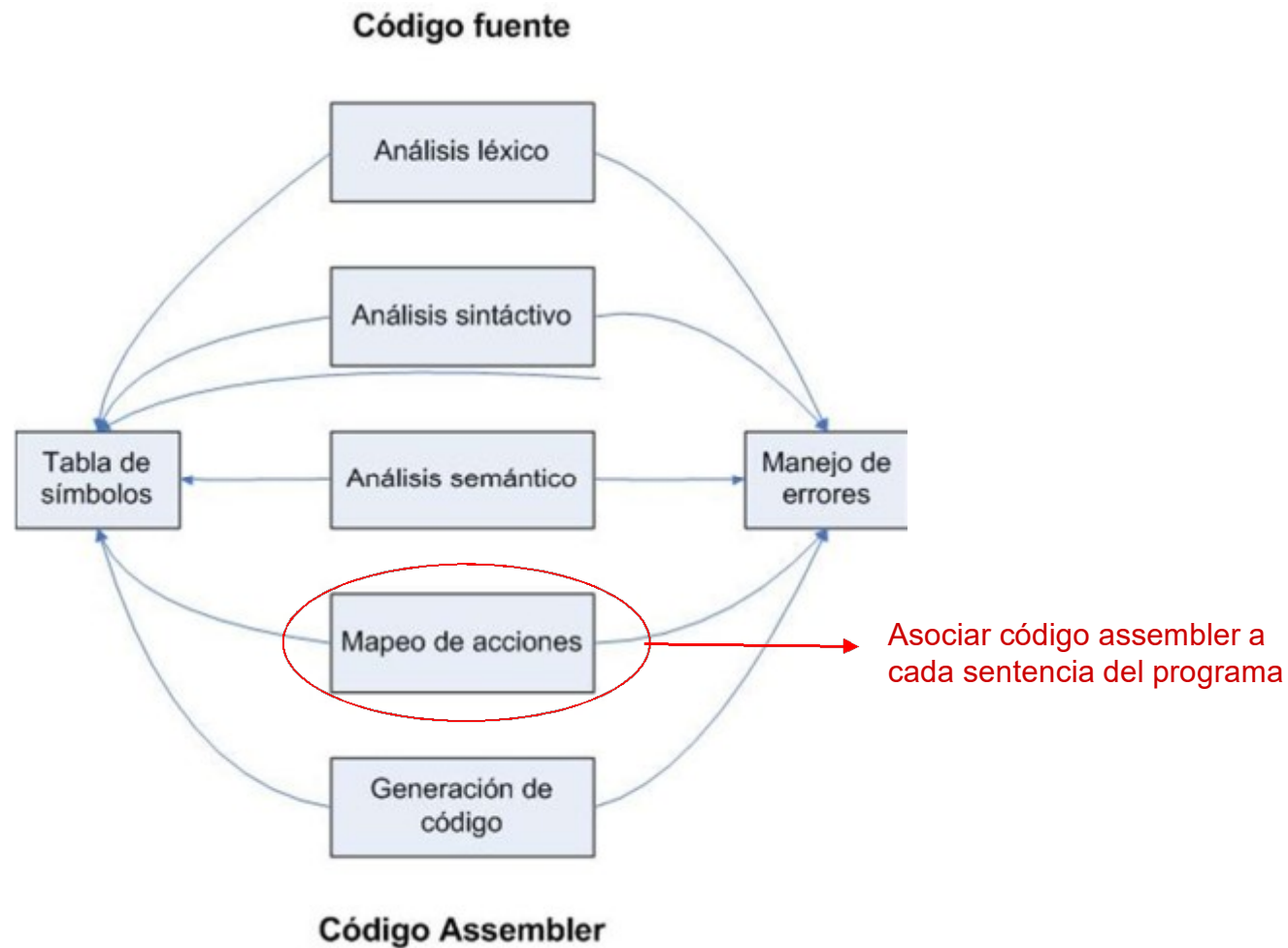
Total = Precio * 5

id1 = id2 * Cte

asignación: id1 "=" expresión
Producto:: "expresión" = id2*Cte

Tabla de Símbolos:
atributos de cada símbolo

El proceso de compilación



Total = Precio * 5

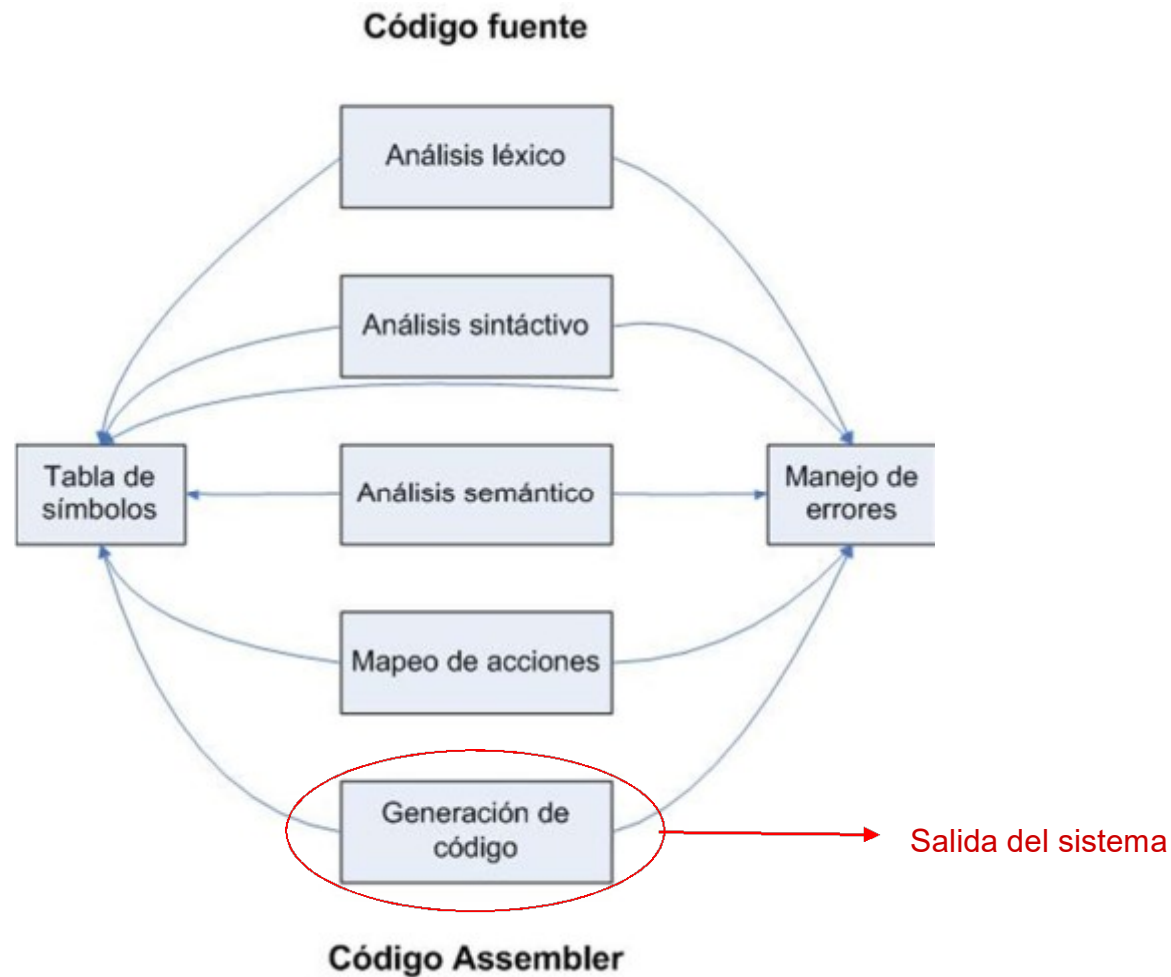
$$\text{id1} = \text{id2} * \text{Cte}$$

asignación: id1 "=" expresión
Producto:: "expresión" = id2*Cte

Tabla de Símbolos:
atributos de cada símbolo

- Código Assembler
- Localiz de las variables.
- Uso de registros

El proceso de compilación



Total = Precio * 5

id1 = id2 * Cte

asignación: id1 "=" expresión
Producto:: "expresión" = id2*Cte

Tabla de Símbolos:
atributos de cada símbolo

- Código assembler
- Variables en registros
- Uso de registros

Mapeo de Acciones



Tres tipos de instrucciones:

- Operaciones aritméticas/lógicas
- Control de flujo del programa
- Movimiento de datos

Mapeo de Acciones

Operaciones aritméticas/lógicas

“ $A=B+4$ ”

! Sentencia de asignación

ld [B], %r1

! guardar variable B en un registro

add %r1, 4, %r2

! calcular el valor de la expresión

st %r2, [A]

! hacer la asignación del valor

- ❖ Modos de direccionamiento a los operadores dependen del ISA
- ❖ En máquinas RISC: operandos siempre en registros
 - Cantidad de registros
 - Cuando es excedida => registros al stack
 - El compilador debe decidir cuáles registros contienen valores que ya no son necesarios.

Requiere de técnicas sofisticadas
¿Porqué se justifica utilizarlas?

Mapeo de Acciones

Estructuras de control de flujo del programa

Goto statement	ba label
if A=B stmt1 else stmt2;	subcc %r1, %r2, %r0 ! setea flags, descarta resultado bne else1 !... código de stmt1 ba fin else1: !código de stmt2 fin: ! ...
while (r1 == r2) %r3 = %r3 + 1;	ba Test True: add %r3, 1, %r3 Test: subcc %r1, %r2, %r0 be True
Do r3 =r3 + 1 while (r1 == r2)	True: add %r3, 1, %r3 Test: subcc %r1, %r2, %r0 be True

Mapeo de Acciones

Variables estructuradas

Estructura	Array en C	Array en Pascal
<pre>struct point { int x; int y; int z; }</pre> <pre>struct point pt;</pre>	<pre>int A[10];</pre> <pre>A[i]</pre>	<pre>A: array [-10..10] of integer</pre> <pre>A[i]</pre>
	<pre>ElementAddress = BASE + INDEX * SIZE</pre>	<pre>ElementAddress = BASE + (INDEX - START) * SIZE</pre>
<pre>ld [pt + 4],%r1! pt.y</pre> 	<pre>SIZE=4, INDEX in %r6</pre> <pre>sll %r6,2,%r6 !%r6 * 4→%r6</pre> <pre>ld [A + %r6],%r1 !valor→%r1</pre> 	<pre>SIZE=4, INDEX en %r3, START en %r4</pre> <pre>sub %r3, %r4, %r6 !INDEX-START → %r6</pre> <pre>sll %r6, 2, %r6 ! %r6 * 4 → %r6</pre> <pre>ld [A + %r6], %r1 ! valor → %r1</pre>

Puntero dentro de la estructura es conocido en tiempo de ensamblado

Puntero dentro de la estructura puede ser conocido en tiempo de ensamblado o en tiempo de ejecución

Mapeo de Acciones

Dónde almacenar variables en RAM

- ❖ Variables estáticas (globales)

Permanecen a lo largo del tiempo de ejecución de la aplicación

=> se guardan en posición de memoria conocida en tiempo de compilación

- ❖ Variables locales a un procedimiento

Desaparecen cuando termina el procedimiento en el que están declaradas

=> se guardan en el stack

- ❖ Guardar variables en registros es óptimo!

Ensamblador



❖ Transcodificación: Relación 1 a 1 entre código Assembler y código de máquina
(Comparar con compilación)

❖ Ofrece al programador

- Representación simbólica para direcciones y constantes
- Definir la ubicación de las variables en memoria
- Variables inicializadas antes de ejecución
- Provee cierto grado de aritmética en tiempo de ensamblado
- Utilizar variables declaradas en otros módulos
- Utilizar macros (dar nombre a fragmentos de texto)

Directivas al ensamblador (ARCTools)

- Indican al ensamblador como procesar una sección de programa
- -Las instrucciones son específicas de un procesador
-Las pseudo-instrucciones o directivas son específicas de un programa ensamblador
- Algunas generan información en la memoria, otras no

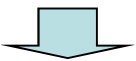
Pseudo-Op	Usage	Meaning
<code>.equ</code>	<code>X .equ #10</code>	Treat symbol X as $(10)_{16}$
<code>.begin</code>	<code>.begin</code>	Start assembling
<code>.end</code>	<code>.end</code>	Stop assembling
<code>.org</code>	<code>.org 2048</code>	Change location counter to 2048
<code>.dwb</code>	<code>.dwb 25</code>	Reserve a block of 25 words
<code>.global</code>	<code>.global Y</code>	Y is used in another module
<code>.extern</code>	<code>.extern Z</code>	Z is defined in another module
<code>.macro</code>	<code>.macro M a, b, ...</code>	Define macro M with formal parameters a, b, ...
<code>.endmacro</code>	<code>.endmacro</code>	End of macro definition
<code>.if</code>	<code>.if <cond></code>	Assemble if <cond> is true
<code>.endif</code>	<code>.endif</code>	End of .if construct

Ensamblador

Obtención del código de máquina

```
! This program adds two numbers
.begin
.org 2048
main: ld    [x], %r1      ! Load x into %r1
      ld    [y], %r2      ! Load y into %r2
      addcc %r1, %r2, %r3 ! %r3 ← %r1 + %r2
      st    %r3, [z]      ! Store %r3 into z
      jmp1  %r15 + 4, %r0 ! Return
x:    15
y:    9
z:    0
.end
```

```
ld    [x], %r1
ld    [y], %r2
```



```
11 00001 000000 00000 1 0100000010100
op  rd  op3  rs1  i  simm13

11 00010 000000 00000 1 0100000011000
op  rd  op3  rs1  i  simm13
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	1																														
rd		op3				rs1				0				rs2																	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	1																														
rd		op3				rs1				1				simm13																	

Formato de acceso a memoria: $op=11$

$op3$	direcc. de memoria (ld o st)	
000000 ld rd=reg.destino	$= rs1 + rs2$	si i = 0
000100 st rd=reg.origen	$= rs1 + \text{simm13 (constante)}$	si i = 1

Proceso de ensamblado en dos pasadas

❖ Preproceso: expansión de macros => (1) Registrar definiciones (2) Reemplazar

❖ Primer pasada

- Detecta identificadores y les asigna una posición de memoria
- Crea la tabla de símbolos

❖ Segunda pasada

- Cada instrucción es convertida a código de máquina
- Cada identificador es reemplazado por su ubicación en memoria según indica la tabla de símbolos
- Cada línea es procesada completamente antes de avanzar a la siguiente
- Genera el código objeto y el listado

(Se completa en la primera pasada)

```

! This program sums LENGTH numbers
! Register usage:          %r1 - Length of array a
!                          %r2 - Starting address of array a
!                          %r3 - The partial sum
!                          %r4 - Pointer into array a
!                          %r5 - Holds an element of a

        .begin                ! Start assembling
        .org 2048             ! Start program at 2048
a_start .equ 3000              ! Address of array a

        ld [length], %r1      ! %r1 ← length of array a
        ld [address], %r2     ! %r2 ← address of a
        andcc %r3, %r0, %r3   ! %r3 ← 0
loop:   andcc %r1, %r1, %r0     ! Test # remaining elements
        be done               ! Finished when length=0
        addcc %r1, -4, %r1     ! Decrement array length
        addcc %r1, %r2, %r4    ! Address of next element
        ld %r4, %r5           ! %r5 ← Memory[%r4]
        addcc %r3, %r5, %r3    ! Sum new element into r3
        ba loop               ! Repeat loop.

done:   jmp %r15 + 4, %r0      ! Return to calling routine

length:                20      ! 5 numbers (20 bytes) in a
address:                a_start
        .org a_start          ! Start of array a
a:                25          ! length/4 values follow
                -10
                33
                -5
                7

        .end                  ! Stop assembling

```

Símbolo	Valor
a_start	3000
length	-
address	-
loop	2060
done	-

1er pasada del assembler

(2)

Símbolo	Valor
a_start	3000
length	2092
address	2096
loop	2060
done	2088

- ❖ Contador de posición (análogo a un *program counter* en tiempo de ensamblado)
 - Inicializado a cero al inicio de la primera pasada y por cada directiva .org
 - Incrementado por cada instrucción según su tamaño (en ARC 4 bytes)

Archivos creados por el ensamblador

(Se completa en la [segunda pasada](#))

Salidas

- ➔ 1) Archivo de texto (listado)
- 2) Archivo con código objeto

Forma parte del
archivo de listado

Símbolo	Valor
a_start	3000
length	2092
address	2096
loop	2060
done	2088

Tabla de símbolos

Location counter	Instruction	Object code
	.begin	
	.org 2048	
	a_start .equ 3000	
2048	ld [length],%r1	11000010 00000000 00101000 00101100
2052	ld [address],%r2	11000100 00000000 00101000 00110000
2056	andcc %r3,%r0,%r3	10000110 10001000 11000000 00000000
2060 loop:	andcc %r1,%r1,%r0	10000000 10001000 01000000 00000001
2064	be done	00000010 10000000 00000000 00000110
2068	addcc %r1,-4,%r1	10000010 10000000 01111111 11111100
2072	addcc %r1,%r2,%r4	10001000 10000000 01000000 00000010
2076	ld %r4,%r5	11001010 00000001 00000000 00000000
2080	ba loop	00010000 10111111 11111111 11111011
2084	addcc %r3,%r5,%r3	10000110 10000000 11000000 00000101
2088 done:	jmp1 %r15+4,%r0	10000001 11000011 11100000 00000100
2092 length:	20	00000000 00000000 00000000 00010100
2096 address:	a_start	00000000 00000000 00001011 10111000
	.org a_start	
3000 a:	25	00000000 00000000 00000000 00011001
3004	-10	11111111 11111111 11111111 11110110
3008	33	00000000 00000000 00000000 00100001
3012	-5	11111111 11111111 11111111 11111011
3016	7	00000000 00000000 00000000 00000111
	.end	

Listado

Archivos creados por el ensamblador

(Se completa en la [segunda pasada](#))

Salidas

- 1) Archivo de texto (listado)
- 2) Archivo con código objeto



- Código de máquina
- Dirección de primera instrucción a ejecutar (si corresponde): *main()*
- Símbolos declarados en otros módulo: *externos*
- Símbolos globales: *accesibles desde otros módulos*
- *Librerías* externas que son utilizadas por el módulo
- Información sobre la *relocalización* del código
- Contenido de la «tabla de símbolos»

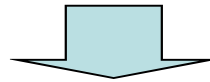
Tipo de ensamblador(

- ❖ ¿Podría hacerse un ensamblador de sólo una pasada ?
- ❖ ¿Sería este funcionamiento notado por el programador?

Localización del programa en memoria

- En general no se sabe donde va a ser cargado el programa
=> ~~“.org 2048”~~
- Si varios módulos son vinculados no se sabe donde va a ser cargado el programa ni cada módulo

~~=> “.org 2048”~~



Código relocizable

- El ensamblador es responsable de marcar direcciones relocizables y direcciones absolutas
- Esta información es necesaria por el linker

EL LINKER

- ✓ Combina dos o más módulos que fueron ensamblados separadamente

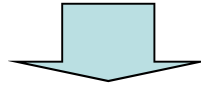
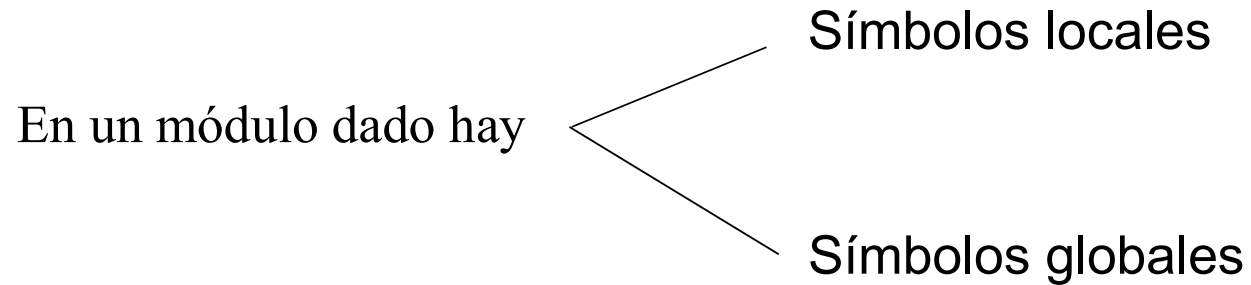
Origen de estos módulos:

- Aplicación
- librería del ambiente de desarrollo
- sistema operativo

- ✓ El linker:

- Resuelve referencias de memoria externa al módulo
- Relocaliza los módulos combinándolos y reasignando las direcciones internas a cada uno para reflejar su nueva localización
- Define en el módulo a cargar la dirección de la primer instrucción a ser ejecutada (“main”)

Referencias externas



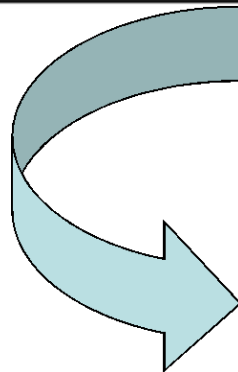
Directivas al ensamblador

<i>.global</i>	→	declara un símbolo global
<i>.extern</i>	→	utiliza símbolo declarado en otro módulo

Declarar una cláusula ".equ" como global no tiene sentido. ¿Porqué?

Referencias externas

<pre>! Main program .begin .org 2048 .extern sub main: ld [x], %r2 ld [y], %r3 call sub jmp1 %r15 + 4, %r0 x: 105 y: 92 .end</pre>	<pre>! Subroutine library .begin ONE .equ 1 .org 2048 .global sub sub: ornc %r3, %r0, %r3 addcc %r3, ONE, %r3 jmp1 %r15 + 4, %r0 .end</pre>
--	---



Symbol	Value	Global/ External
sub	-	External
main	2048	No
x	2064	No
y	2068	No

Main Program

Symbol	Value	Global/ External
ONE	1	No
sub	2048	Global

Subroutine Library

INCOMPLETAS
(sigue en próx transparencia)

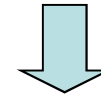
Símbolos relocizables

TIEMPO DE ENSAMBLADO

<pre>! Main program .begin .org 2048 .extern sub main: ld [x], %r2 ld [y], %r3 call sub jmp %r15 + 4, %r0 x: 105 y: 92 .end</pre>	<pre>! Subroutine library .begin ONE .equ 1 .org 2048 .global sub sub: ornc %r3, %r0, %r3 addcc %r3, ONE, %r3 jmp %r15 + 4, %r0 .end</pre>
--	--

TIEMPO DE LINKING

~~Addr(Main) = Addr(Sub) = 2048~~



Direcciones relocizables

- Ensamblador

Marca direcciones como relocizables o no relocizables (absolutas)

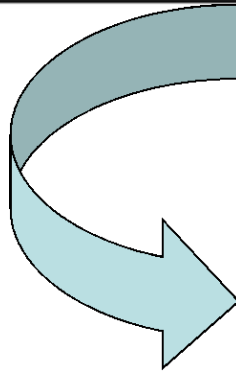
- Linker

Redefine las direcciones relocizables a partir de la nueva dirección de origen

Símbolos relocalizables

```
! Main program
    .begin
    .org 2048
    .extern sub
main: ld    [x], %r2
      ld    [y], %r3
      call  sub
      jmp1  %r15 + 4, %r0
x: 105
y: 92
    .end
```

```
! Subroutine library
    .begin
ONE .equ 1
    .org 2048
    .global sub
sub: ornc  %r3, %r0, %r3
      addcc %r3, ONE, %r3
      jmp1  %r15 + 4, %r0
    .end
```



Symbol	Value	Global/ External	Reloc- atable
sub	-	External	-
main	2048	No	Yes
x	2064	No	Yes
y	2068	No	Yes

Main Program

Symbol	Value	Global/ External	Reloc- atable
ONE	1	No	No
sub	2048	Global	Yes

Subroutine Library

Símbolos relocizables

EL ENSAMBLADOR

- ✓ Determina cuáles símbolos son relocizables y cuáles no
- ✓ Los marca en el módulo ensamblado
- ✓ Identifica código que debe ser modificado como resultado de la relocización

> No tiene sentido marcar como relocizables símbolos declarados en otros módulos

No son relocizables:

- ✓ *Direcciones de entrada/salida*
- ✓ *Rutinas del sistema*

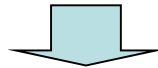
Sí son relocizables:

- ✓ Posiciones de memoria relativas a un .org (p.e.: x o y)

Carga del programa en memoria

*El loader toma el programa de disco y lo carga en memoria principal
¿en qué dirección de memoria?*

- En ambientes multitarea la ram es compartida entre varios proceso
- El assembler no puede saber donde puede ser cargado
- El linker no puede saber donde puede ser cargado



El loader debe relocar todos los símbolos relocizables

Ajustando la definición de “linker” y de “loader”

■ Link-editor

–Produce una versión linkeada del programa que normalmente es guardada en archivo para ser ejecutada en otro momento

■ Linking loader

–Linkea en tiempo de carga.

–Relocaliza, busca automáticamente librerías, linkea y carga el programa en memoria para su ejecución

...

Ajustando la definición de “linker” y de “loader”

■ Link-editor

–Produce una versión linkeada del programa que normalmente es guardada en archivo para ser ejecutada en otro momento

■ Linking loader

–Linkea en tiempo de carga.

–Relocaliza, busca automáticamente librerías, linkea y carga el programa en memoria para su ejecución

■ Linking Loader dinámico

–Linkea en tiempo de ejecución

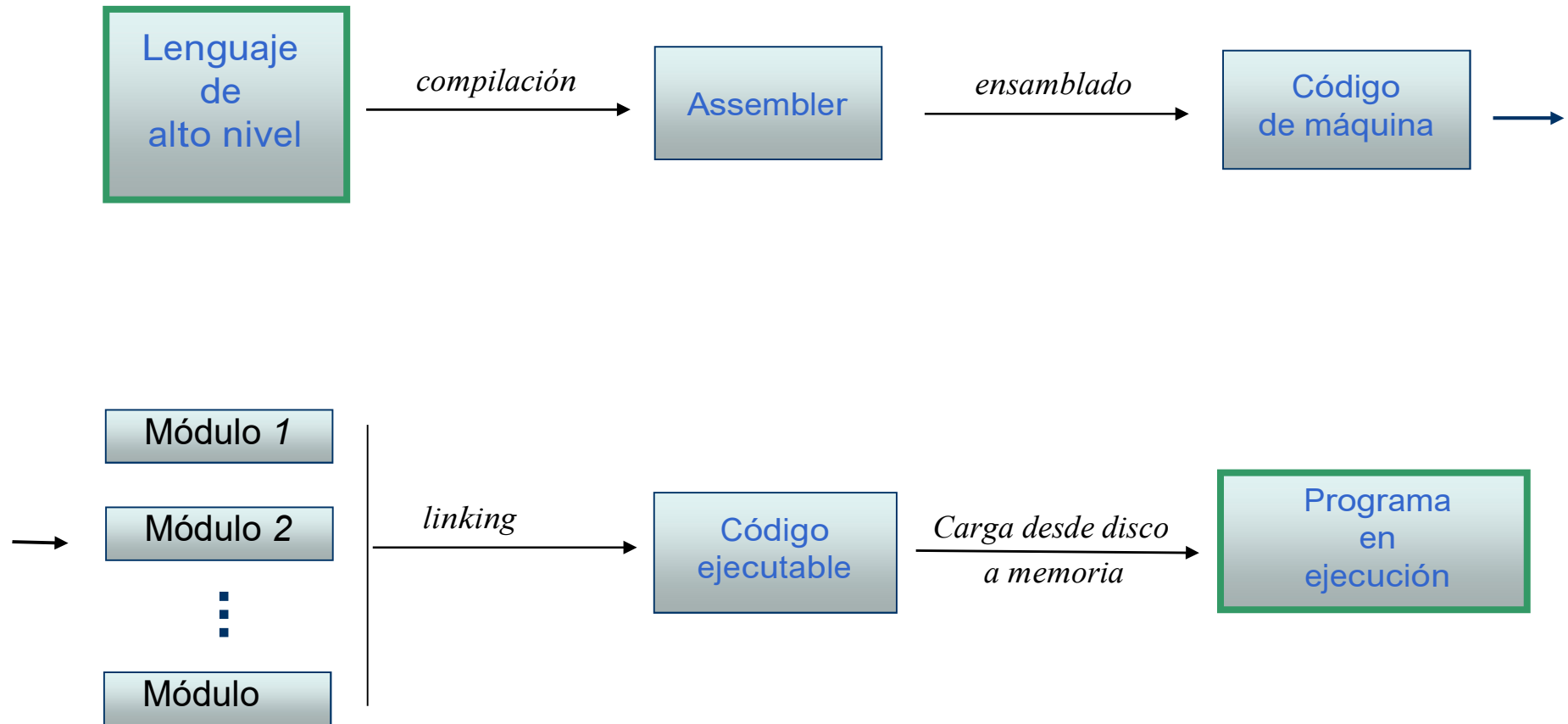
–Utiliza Dynamic Link Library

–Carga rutinas sólo cuando el programa las necesita

Archivos objeto

- ✓ Archivo objeto *relocateable* : código binario y datos en un formato que permite combinarlo con otros archivos objeto relocateables (linking)
 - ✓ Archivo objeto *ejecutable*: código binario y datos en un formato que permite cargarlo directamente a memoria y ejecutarlo
 - ✓ Archivo objeto *compartido*: tipo especial de archivo objeto relocateable que puede ser cargado en memoria y vinculado dinámicamente
-
- El formato de los archivos objeto es dependiente del sistema donde van a ejecutarse

Desde el diseño a la ejecución



El proceso de ensamblado

El proceso de traducción de un programa expresado en lenguaje simbólico se conoce como **proceso de ensamblado**. Este proceso es lineal y relativamente simple, dado que hay una relación directa y univoca entre las sentencias del lenguaje simbólico y sus correspondientes expresiones en el lenguaje de máquina. Esto marca una diferencia con el proceso de compilación, por ejemplo, en el que una sentencia de un lenguaje de alto nivel dado puede traducirse como una buena cantidad de sentencias en el lenguaje de máquina.

Si bien el ensamblado es un proceso lineal, es tedioso y proclive al error si se hace a mano. De hecho, el programa ensamblador es una de las primeras herramientas de software desarrolladas luego de la invención de la computadora electrónica digital.

Los ensambladores comerciales deben ofrecer, al menos, las siguientes prestaciones:

- Permitir que el programador especifique la ubicación de las variables y programas al momento de la ejecución. (Sin embargo, en la mayoría de los casos el programador no especifica una dirección absoluta de comienzo de un programa, dado que el programa puede ser movido o reubicado por el programa vinculador (*linker*) y, probablemente, por el programa de carga (*loader*), de acuerdo con lo que se analizará más adelante.)
- Ofrecerle al programador la posibilidad de inicializar los valores de los datos en memoria antes de la ejecución del programa.
- Proveer expresiones nemotécnicas en el lenguaje de programación para todas las instrucciones del lenguaje de máquina y modos de direccionamiento, y traducir las

sentencias válidas del lenguaje simbólico hacia sus valores binarios equivalentes en el lenguaje absoluto.

- Permitir el uso de rótulos simbólicos para identificar o representar direcciones y constantes.
- Ofrecerle al programador una forma de especificar la dirección de comienzo de un programa, si existiera. (Por ejemplo, no habría dirección de comienzo si el módulo que se traduce es una función o un procedimiento.)
- Ofrecer cierto grado de cálculo al momento del ensamblado.
- Incluir un mecanismo que permita la definición de variables en un programa escrito en lenguaje simbólico y el uso de las mismas en otro programa ensamblado por separado.
- Proveer la expansión de **macro rutinas**, o sea, rutinas que puedan definirse una vez y utilizarse tantas veces como sea necesario.

Como técnica general, el proceso de ensamblado se lleva a cabo leyendo las sentencias del lenguaje simbólico en forma secuencial, desde la primera hasta la última, y generando el código de máquina para cada una de las sentencias.

En la primera pasada de un proceso de ensamblado de dos pasadas se crea una **tabla de símbolos**. Un símbolo puede ser un rótulo o un nombre simbólico que se refiera a un valor utilizado durante el proceso de ensamblado.

El proceso de ensamblado y los ensambladores de dos pasadas

La mayoría de los ensambladores recorren dos veces el texto escrito en lenguaje simbólico, por lo que se los conoce como “ensambladores de dos pasadas”. En la primera pasada el ensamblador se dedica a determinar las direcciones de todos los datos e instrucciones del programa y a seleccionar qué instrucción del lenguaje de máquina debe generarse para cada instrucción del lenguaje simbólico, pero sin generar aún el código de máquina.

Las direcciones de los datos y de las instrucciones se determinan en el momento del ensamblado mediante un contador similar al contador de programa, al que se conoce como **contador de posición** (*location counter*). A medida que se desarrolla el proceso de ensamblado, este contador de posición lleva el control de la dirección de la instrucción o del elemento de datos corriente. Generalmente, se lo inicializa a 0 al comienzo del primer paso y se lo incrementa en pasos equivalentes al tamaño de cada instrucción. La directiva **.org** hace que el contador se ubique en el valor especificado por la misma. Por ejemplo, en caso de encontrar la sentencia

```
.org 1000
```

el contador de posición adoptará el valor 1000, por lo que la instrucción o dato siguiente se asignará a dicha dirección. Durante esta pasada, el ensamblador realiza también cualquier operación aritmética necesaria e inserta las definiciones de todos los rótulos y constantes en una **tabla de símbolos**.

La razón principal para requerir una segunda pasada tiene que ver con la necesidad de permitir el uso de los símbolos dentro de un programa con anterioridad a que sean definidos, lo que se suele conocer como **referencia previa**. Luego de la primera pasada, el ensamblador tendrá todos sus símbolos definidos e ingresados en la tabla de símbolos,

por lo que durante la segunda pasada podrá generar el código de máquina, insertando en el mismo los valores de los símbolos, ya conocidos para ese momento.

